Examen Parcial

Pedro Omar Casabella Lemus

October 2026

Ejercicio de Listas Enlazadas

Filtrado de pacientes en el área de Urgencias

Documento técnico de la solución

Lenguaje:	Java
Estructura:	Lista simplemente enlazada
Método clave:	`obtenerPacientesEnUrgencias()`
Entorno:	Visual Studio Code + JDK

26 de octubre de 2026

Índice

1. Planteamiento del problema **2**

2. Diseño de la solución **2**
2.1. Análisis del enfoque inicial (descartado) . . . 2
2.2. Diseño adoptado . . . 2

3. Diagramas de flujo **3**
3.1. Flujo general de la aplicación . . . 3
3.2. Flujo del método `obtenerPacientesEnUrgencias()` . . . 3
3.3. Flujo de `agregar(Paciente p)` . . . 5

4. Código fuente **5**
4.1. Paciente.java . . . 5
4.2. Nodo.java . . . 6
4.3. ListaPacientes.java . . . 6
4.4. Main.java . . . 7

5. Casos de prueba **7**

6. Justificación técnica de las decisiones **8**

7. Conclusiones **8**

1. Planteamiento del problema

Se dispone de una clase `Paciente` con los atributos `id`, `nombre`, `edad`, `areaAtencion` y `diasInternado`. Se pide:

> *Recorrer la lista simplemente enlazada de pacientes y construir una nueva lista que contenga únicamente los pacientes cuyo atributo `areaAtencion` sea igual a `Ürgencias"`. Si no existen pacientes en dicha área, el método debe retornar una lista vacía sin producir excepciones de puntero nulo (`NullPointerException`).*

Requisitos explícitos

R1. Recorrer una lista *simplemente enlazada* (no un arreglo ni un `ArrayList`).

R2. Construir una **nueva** lista; no modificar la original.

R3. Filtrar por igualdad exacta con el literal `Ürgencias"`.

R4. Si no hay coincidencias, devolver lista vacía (no `null`).

R5. No lanzar `NullPointerException` en ningún caso.

2. Diseño de la solución

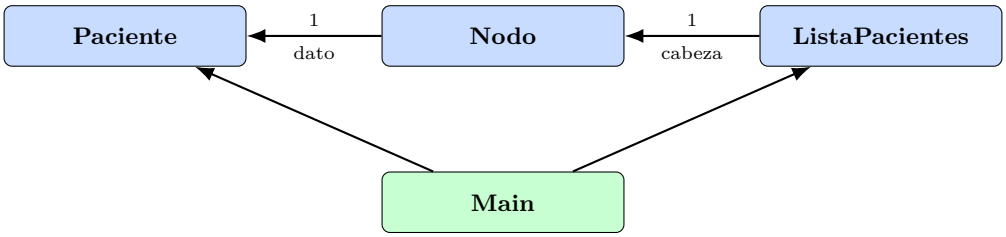
2.1. Análisis del enfoque inicial (descartado)

El primer intento consistió en declarar `public class LinkedList extends Paciente`. Este enfoque es *incorrecto* por dos razones:

- `extends` modela una relación *«es un»*. Una lista **no es** un paciente: lo **contiene**. La relación correcta es de **composición**.
- Una lista enlazada requiere **nodos** con un dato y una referencia al siguiente. Al heredar de `Paciente` la clase solo tendría los campos de *un* paciente, sin estructura para encadenar varios.

2.2. Diseño adoptado

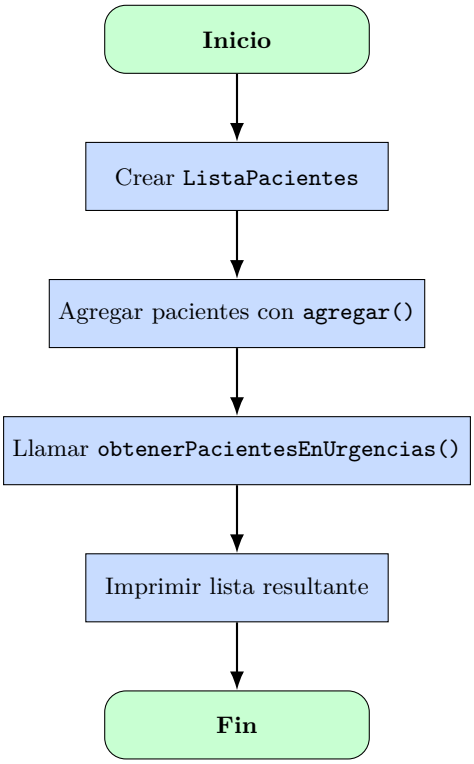
Se separan las responsabilidades en cuatro clases:



- **Paciente**: entidad del dominio.
- **Nodo**: eslabón con `Paciente dato` y `Nodo siguiente`.
- **ListaPacientes**: contiene `Nodo cabeza` y los métodos `agregar()`, `obtenerPacientesEnUrgencias()`, etc.
- **Main**: punto de entrada con los casos de prueba.

3. Diagramas de flujo

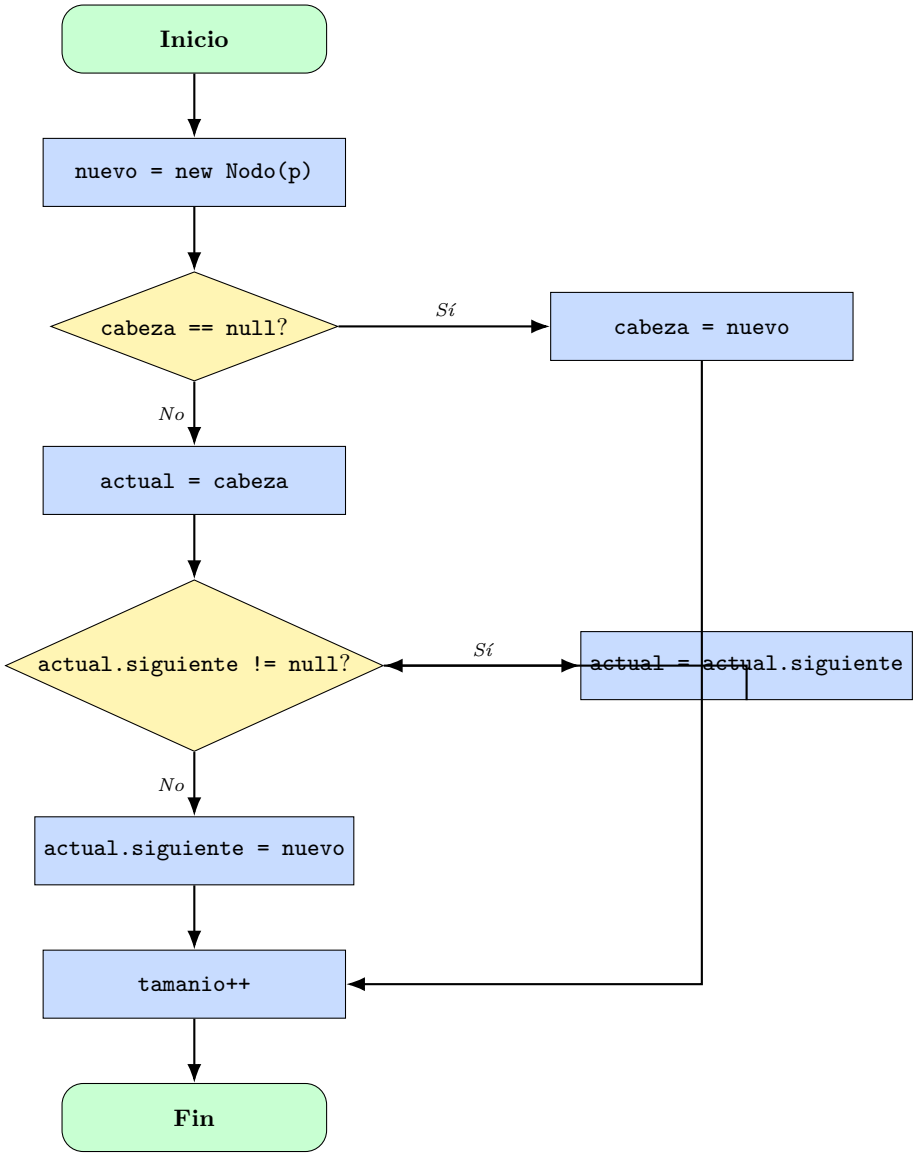
3.1. Flujo general de la aplicación



3.2. Flujo del método `obtenerPacientesEnUrgencias()`

Este es el corazón del ejercicio. El diagrama muestra la defensa contra `NullPointerException` y la construcción de la lista nueva.

3.3. Flujo de agregar(Paciente p)



4. Código fuente

4.1. Paciente.java

```java
public class Paciente {
    public String id;
    public String nombre;
    public int edad;
    public String areaAtencion;
    public int diasInternado;

    public Paciente(String id, String nombre, int edad,
                    String areaAtencion, int diasInternado) {
        this.id = id;
        this.nombre = nombre;
        this.edad = edad;
        this.areaAtencion = areaAtencion;
```

```
        this.diasInternado = diasInternado;
    }

    @Override
    public String toString() {
        return "Paciente{id='" + id + "', nombre='" + nombre +
               "', areaAtencion='" + areaAtencion + "'}";
    }
}
```

Listing 1: Entidad del dominio

4.2. Nodo.java

```
public class Nodo {
    Paciente dato;
    Nodo siguiente;

    public Nodo(Paciente dato) {
        this.dato = dato;
        this.siguiente = null;
    }
}
```

Listing 2: Eslabón de la lista simplemente enlazada

4.3. ListaPacientes.java

```
public class ListaPacientes {
    private Nodo cabeza;
    private int tamanio;

    public ListaPacientes() {
        this.cabeza = null;
        this.tamanio = 0;
    }

    public void agregar(Paciente p) {
        Nodo nuevo = new Nodo(p);
        if (cabeza == null) {
            cabeza = nuevo;
        } else {
            Nodo actual = cabeza;
            while (actual.siguiente != null) {
                actual = actual.siguiente;
            }
            actual.siguiente = nuevo;
        }
        tamanio++;
    }

    public int getTamanio() { return tamanio; }
    public boolean estaVacia() { return cabeza == null; }

    public ListaPacientes obtenerPacientesEnUrgencias() {
        ListaPacientes resultado = new ListaPacientes();
        Nodo actual = cabeza;
        while (actual != null) {
            Paciente p = actual.dato;
            if (p != null && "Urgencias".equals(p.areaAtencion)) {
                resultado.agregar(p);
            }
            actual = actual.siguiente;
        }
```

```
        return resultado;
    }

    public void imprimir() {
        if (cabeza == null) {
            System.out.println("  (lista vacia)");
            return;
        }
        Nodo actual = cabeza;
        while (actual != null) {
            System.out.println("  " + actual.dato);
            actual = actual.siguiente;
        }
    }
}
```

Listing 3: Lista y método del ejercicio

4.4. Main.java

```
public class Main {
    public static void main(String[] args) {
        // Caso 1: lista variada
        ListaPacientes lista = new ListaPacientes();
        lista.agregar(new Paciente("1","Ana",30,"Urgencias",2));
        lista.agregar(new Paciente("2","Luis",45,"Cardiologia",5));
        lista.agregar(new Paciente("3","Eva",22,"Urgencias",1));
        lista.agregar(new Paciente("4","Mario",60,"Urgencias",7));
        lista.agregar(new Paciente("5","Sofia",28,"Pediatria",3));

        ListaPacientes u = lista.obtenerPacientesEnUrgencias();
        System.out.println("En Urgencias: " + u.getTamanio());
        u.imprimir();
    }
}
```

Listing 4: Punto de entrada con casos de prueba

5. Casos de prueba

La siguiente tabla resume los escenarios ejecutados y su resultado.

Caso	Escenario	Entrada	Resultado esperado	OK
1	Lista con 5 pacientes variados	5 pacientes	3 en Urgencias	
2	Lista original vacía	0 pacientes	Lista vacía, sin excepción	
3	Pacientes pero ninguno en Urgencias	2 pacientes	Lista vacía	
4	Paciente con `areaAtencion = null`	2 pacientes (1 null)	1 en Urgencias, sin NPE	

Salida real de la consola (Caso 1)

```
Lista original (5 pacientes):
  Paciente{id='1', nombre='Ana',   areaAtencion='Urgencias'}
  Paciente{id='2', nombre='Luis',  areaAtencion='Cardiologia'}
```

```
  Paciente{id='3', nombre='Eva',   areaAtencion='Urgencias'}
  Paciente{id='4', nombre='Mario', areaAtencion='Urgencias'}
  Paciente{id='5', nombre='Sofia', areaAtencion='Pediatria'}
Pacientes en Urgencias (3):
  Paciente{id='1', nombre='Ana',   areaAtencion='Urgencias'}
  Paciente{id='3', nombre='Eva',   areaAtencion='Urgencias'}
  Paciente{id='4', nombre='Mario', areaAtencion='Urgencias'}
```

Listing 5: Salida de java Main

6. Justificación técnica de las decisiones

¿Por qué Ürgencias".equals(...) y no al revés?

Porque el literal nunca es `null`. Si se escribiera `p.areaAtencion.equals(Ürgencias")` y el atributo fuera `null`, se lanzaría `NullPointerException`. La forma empleada devuelve `false` de manera segura.

¿Por qué se devuelve una lista nueva?

El enunciado pide *construir* una nueva lista. Además, así se respeta el principio de no efectos secundarios: la lista original permanece intacta y puede reutilizarse.

¿Por qué el método no falla si la lista está vacía?

Porque `cabeza == null` implica que el bucle `while` no se ejecuta ni una vez, y se retorna `resultado` (que se creó vacía al inicio). No hay acceso a miembros de un objeto nulo.

Complejidad

El recorrido es de orden $O(n)$ en tiempo, donde n es el número de pacientes. En espacio, la nueva lista contiene a lo sumo n nodos: $O(k)$ con $k \leq n$.

7. Conclusiones

- Se resolvió el ejercicio usando una lista simplemente enlazada con nodos, descartando el enfoque incorrecto de herencia (`LinkedList extends Paciente`).
- El método `obtenerPacientesEnUrgencias()` cumple los cinco requisitos del enunciado.
- Los cuatro casos de prueba demuestran el correcto funcionamiento, incluida la defensa contra `NullPointerException`.
- El proyecto compila con `javac *.java` y se ejecuta con `java Main` en Visual Studio Code.